

บทที่ 2

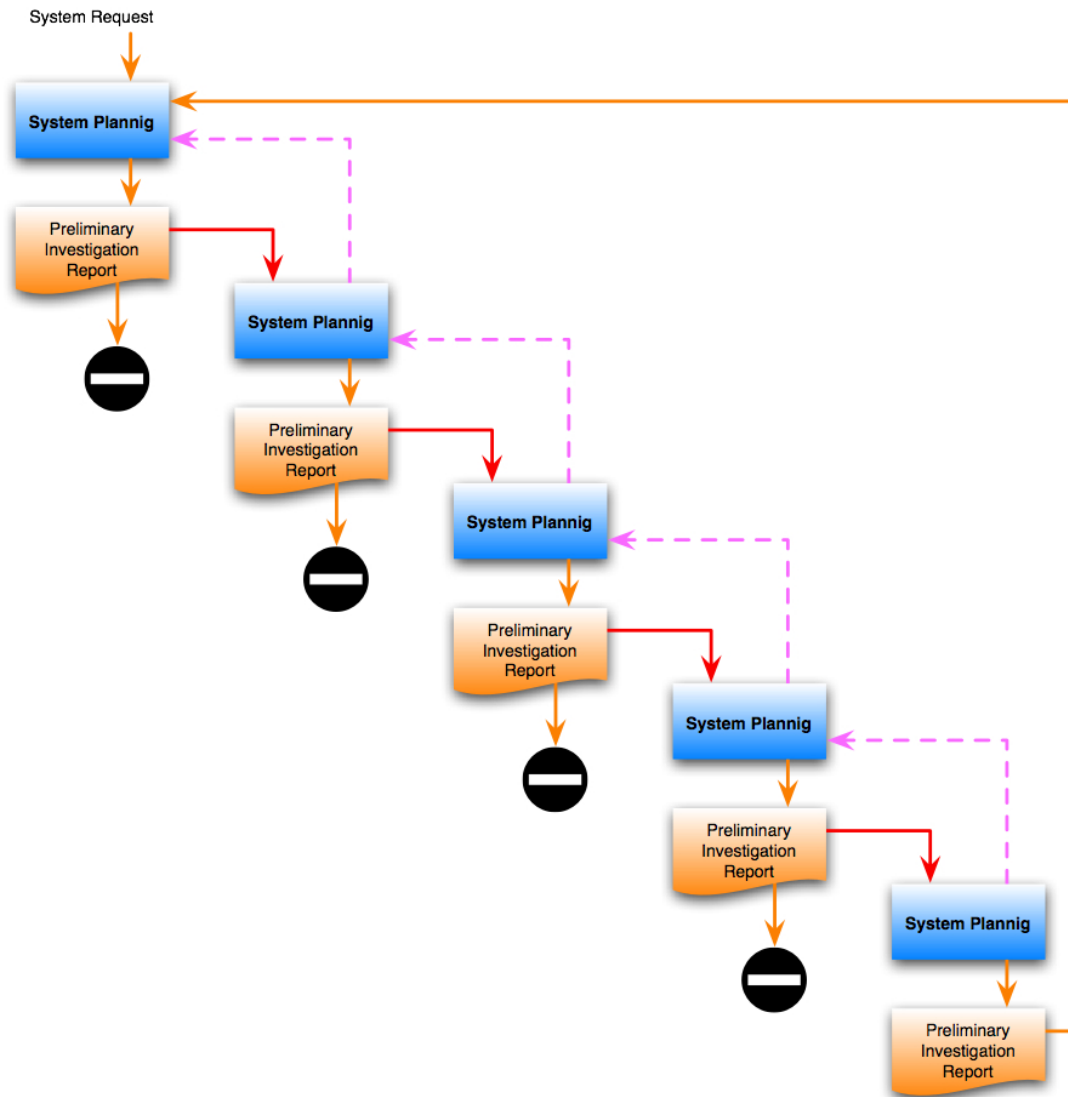
ทฤษฎีพื้นฐานที่ใช้ในโครงการ

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆ ที่เกี่ยวข้องในการทำโครงการ ซึ่งเนื้อหาในบทนี้จะกล่าวถึงทฤษฎีและแนวทางการออกแบบต่างๆดังต่อไปนี้ Software Development Life Cycle, Model-View-Controller Design Pattern, Multi-tier (n-tier) Architecture, Java 2 Enterprise Edition (J2EE) Architecture, Oracle Application Development Framework (OADF) Architecture ซึ่งเนื้อหาเหล่านี้จำเป็นในการออกแบบและพัฒนาระบบตามแนวทางการออกแบบและพัฒนาที่ทาง ORACLE แนะนำ

2.1 System Development Life Cycle (SDLC)

System Development Life Cycle (SDLC) เป็นกระบวนการในการพัฒนา ดูแลรักษาหรือเปลี่ยนแปลงระบบสารสนเทศ นอกจากนี้ SDLC ยังรู้จักกันในชื่อของ Information System Development หรือ Application Development อีกด้วย SDLC อธิบายกิจกรรมและหน้าที่ซึ่งผู้พัฒนาระบบจะต้องทำโดยไม่คำนึงถึงวิธีการที่ใช้ ประกอบไปด้วยขั้นตอนหลักๆ 5 ขั้นตอนดังต่อไปนี้ System Planning, System Analysis, System Design, System Implementation, System Operation, Support and Security

SDLC นั้นมีการอ้างอิงถึงในแบบจำลองที่หลากหลาย โดยมี Waterfall Model เป็นแบบจำลองดั้งเดิมซึ่งใน Waterfall Model จะแสดงให้เห็นว่าผลลัพธ์จากแต่ละขั้นตอนจะไหลไปสู่ขั้นตอนต่อไปเป็นอย่างเป็นลำดับ ดังรูปที่ 2.1



รูปที่ 2.1 Waterfall Model ใน System Development Life Cycle

2.1.1 การวางแผนระบบ (System Planning)

การวางแผนระบบเป็นขั้นตอนในการวางแผน โดยมีวัตถุประสงค์หลักเพื่อการทำการประเมินสิ่งที่ถูกร้องขอเบื้องต้น (Preliminary Investigation) เพื่ออธิบายธรรมชาติและขอบเขตของปัญหาและโอกาสในการสร้างระบบ หรือในธุรกิจ

Preliminary Investigation เป็นขั้นตอนในการศึกษา System Request และ Action โดยมีขั้นตอนดังต่อไปนี้

1. Understand Problems and Opportunities เพื่อศึกษาปัญหาของระบบดั้งเดิม หรือปัญหาของธุรกิจใดๆ แล้วค้นหาโอกาสในการนำเอาระบบที่จะออกแบบและพัฒนาเข้ามาช่วยเพิ่มประสิทธิภาพ และแก้ปัญหของระบบเดิม
2. Define Project Scope เพื่อกำหนดขอบเขตของระบบที่จะออกแบบและพัฒนา

3. Perform Fact Finding เป็นการศึกษาข้อมูลของระบบเดิมหรือธุรกิจ โดยอาจใช้การสัมภาษณ์หรือแบบสอบถาม เพื่อให้เข้าใจการทำงาน และรายละเอียดของระบบหรือธุรกิจนั้นๆมากยิ่งขึ้น
4. Evaluate Feasibility เป็นการศึกษาความเป็นไปได้ของการออกแบบพัฒนาระบบ โดยแบ่งออกเป็น 4 ส่วนได้แก่ ความเป็นไปได้เชิงเทคนิค (Technical Feasibility), ความเป็นไปได้เชิงการทำงาน (Operational Feasibility), ความเป็นไปได้เชิงเวลา (Schedule Feasibility) และความเป็นไปได้ในเชิงเศรษฐศาสตร์ (Economic Feasibility)
5. Estimate Project Development time and cost ทำการประมาณเวลาและค่าใช้จ่ายในการออกแบบ และพัฒนาระบบใหม่ เพื่อกำหนดช่วงเวลาที่แน่นอน
6. Present Results เพื่อแสดงข้อมูลที่ได้ทำการศึกษา และรับฟังความคิดเห็นจากผู้ใช้งาน

โดยมากขึ้นตอนนี้มักเริ่มต้นด้วยการพิจารณาสิ่งที่ถูกร้องขอ (System Request) ซึ่งจะแสดงถึงปัญหา หรือสิ่งที่ต้องการให้มีการเปลี่ยนแปลงในระบบสารสนเทศ หรือกระบวนการทางธุรกิจ และได้ผลลัพธ์สุดท้ายเป็นรายงานการศึกษาเบื้องต้น (Preliminary Investigation Report)

2.1.2 การวิเคราะห์ระบบ (System Analysis)

การวิเคราะห์ระบบ เป็นขั้นตอนในการวิเคราะห์ระบบ โดยมีวัตถุประสงค์หลักเพื่อสร้างแบบจำลองเชิงตรรกะของระบบใหม่ ในขั้นตอนนี้มีขั้นตอนย่อยดังต่อไปนี้

1. Requirement Model เป็นการศึกษาข้อเท็จจริง (Fact Finding) เพื่ออธิบายระบบในปัจจุบัน และแสดงถึงสิ่งที่ต้องการในระบบใหม่ เช่น Input, Output, Process, Control ฯลฯ
2. Data and Process Model เป็นการศึกษาและวิเคราะห์วิธีการในการนำเสนอข้อมูลระบบ และกระบวนการต่างๆของระบบ มีการสร้างแบบจำลองเชิงตรรกะ (Logical Model) ซึ่งจะอธิบายว่าระบบต้องทำอะไรได้บ้าง และแบบจำลองเชิงกายภาพ (Physical Model) ซึ่งจะอธิบายว่าระบบจะถูกสร้างขึ้นได้อย่างไร
3. Development Strategies เป็นการพิจารณาทางเลือกและเตรียมการย้ายไปสู่ขั้นตอนการออกแบบ มีการสร้างตัวอย่างต้นแบบ (Design Prototype) เพื่อให้ผู้ใช้งานนำไปทดสอบและแสดงความคิดเห็น

เมื่อสิ้นสุดขั้นตอนนี้จะได้ผลลัพธ์สุดท้ายเป็น System Requirement ซึ่งอธิบายการบริหารจัดการ ความต้องการของผู้ใช้ ค่าใช้จ่าย ผลประโยชน์ และแนวทางการพัฒนาระบบ

2.1.3 การออกแบบระบบ (System Design)

การออกแบบระบบ เป็นขั้นตอนในการออกแบบระบบ โดยมีวัตถุประสงค์เพื่อสร้างต้นแบบที่ตรงตามความต้องการของระบบทั้งหมด ในขั้นตอนนี้มีขั้นตอนย่อยดังต่อไปนี้

1. การออกแบบ Input/Output และ User Interface ของระบบ โดยจะมุ่งเน้นตอบสนองความต้องการของผู้ใช้งานให้มากที่สุด
2. การออกแบบข้อมูล มุ่งเน้นการออกแบบโครงสร้างข้อมูล และการจัดเก็บข้อมูลให้มีความซับซ้อนน้อยที่สุด
3. การออกแบบสถาปัตยกรรมของระบบ มุ่งเน้นการออกแบบระบบให้มีความยืดหยุ่น มีค่าใช้จ่ายที่เหมาะสม และใช้งานได้ดี

นอกจากนี้ ในขั้นตอนนี้จะมีการแสดง Input, Output และ Process ของระบบ และจะได้ผลลัพธ์สุดท้ายเป็น System Design Specification

2.1.4 การพัฒนาระบบ (System Implementation)

การพัฒนาระบบ เป็นขั้นตอนในการพัฒนาระบบ มีวัตถุประสงค์หลักเพื่อพัฒนาระบบให้สามารถใช้งานได้อย่างสมบูรณ์พร้อมทั้งเอกสารประกอบของระบบที่พัฒนา

ในขั้นตอนนี้ใช้ System Design Specification เป็นแบบในการสร้างระบบใหม่ โดยมีขั้นตอนย่อยดังต่อไปนี้

1. การพัฒนาระบบ เป็นกระบวนการแปลง Program Logic ไปสู่ชุดคำสั่งซึ่งคอมพิวเตอร์สามารถนำไปประมวลผลได้
2. การทดสอบระบบ เป็นกระบวนการเพื่อตรวจสอบ และแก้ไขข้อผิดพลาดในระบบ
 - a) Unit Testing เป็นการแยกทดสอบระบบทีละจุด เพื่อหาข้อบกพร่องในแต่ละจุดแล้วทำการแก้ไข
 - b) Integrating Testing เป็นการทดสอบการทำงานร่วมกัน หลังจากเสร็จสิ้นการทำ Unit Testing แล้ว เพื่อทดสอบการทำงานร่วมกันของระบบ
 - c) System Testing เป็นการทดสอบการทำงานครั้งสุดท้ายหลังจากเสร็จสิ้น Integrating Testing โดยจะทำการทดสอบโดยให้ระบบทำงานจริง แล้วคอยวิเคราะห์ผลการทำงาน
3. การทำเอกสารประกอบ เพื่ออธิบายการทำงานของระบบ และช่วยเหลือผู้ใช้งานที่ต้องทำงานร่วมกับระบบ
4. การฝึกอบรมผู้ใช้งาน เพื่อให้ผู้ใช้งานมีความพร้อม และคุ้นเคยกับระบบ

เมื่อสิ้นสุดขั้นตอนนี้ ระบบจะพร้อมใช้งานได้จริงทันที ซึ่งขั้นตอนสุดท้ายนี้จะรวมไปถึงการแปลงข้อมูลเพื่อนำไปใช้ในระบบใหม่ การอบรมผู้ใช้งาน และการทดลองการทำงานจริง

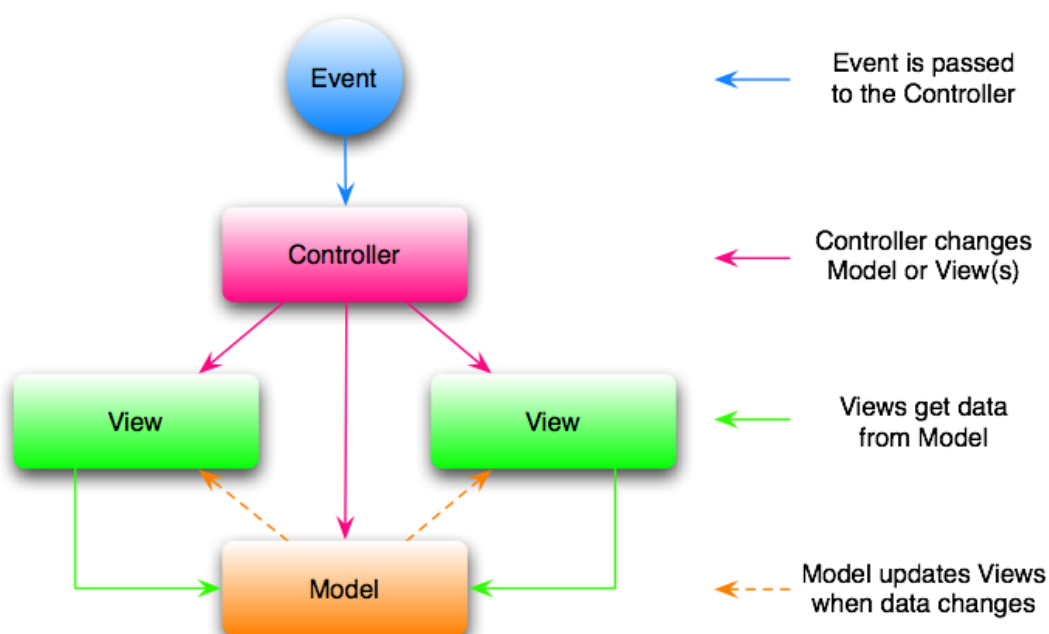
นอกจากนี้ ในขั้นตอนนี้อาจมีการเพิ่มเติมในส่วนของ System Evolution เพื่อตัดสินใจว่าระบบสามารถใช้งานได้มีประสิทธิภาพและมีค่าใช้จ่ายและผลประโยชน์ที่ได้ตามที่คาดการณ์หรือไม่

2.1.5 การนำระบบไปใช้งานจริง (System Operation Support, Security)

การนำเอาระบบไปใช้งานจริง เป็นขั้นตอนในการนำเอาระบบไปใช้งานจริง โดยในขั้นตอนนี้ จะมีการซ่อมบำรุงและแก้ไขข้อผิดพลาด เพิ่มความสามารถและความปลอดภัยให้แก่ระบบ โดยขั้นตอนนี้จะกินระยะเวลาดังแต่ทำการพัฒนาระบบเสร็จสิ้น จนถึงระบบไม่ถูกใช้งานอีกต่อไปแล้ว

2.2 Model-View-Controller Design Pattern

Model-View-Controller Design Pattern เป็นแนวทางการออกแบบ ที่แบ่งแยกการพัฒนา Software ออกจากกันอย่างชัดเจนได้ 3 ส่วน ได้แก่ Data Model, User Interface และ Control Logic ซึ่งจะส่งผลให้เมื่อมีการแก้ไขในส่วนใดๆแล้วจะมีผลกระทบต่อส่วนอื่นน้อยมาก โดยแต่ละส่วนจะมีผู้ควบคุมดูแลดังต่อไปนี้



รูปที่ 2.2 MVC Design Pattern

2.2.1 ชั้นข้อมูล (Model Layer)

Model Layer ควบคุมดูแลในส่วนของ Data Model ทำหน้าที่นำเสนอข้อมูลและจัดเตรียมการเข้าถึงข้อมูลต่างๆให้แก่โปรแกรม บางครั้งอาจเรียก Domain Layer โดยใน Model

Layer นี้จะมีกลไกในการติดต่อ เรียกใช้ และจัดการข้อมูลจากแหล่งข้อมูลที่หลากหลาย เป็นอิสระต่อกัน และ Transparent ต่อ Layer อื่นๆ นั่นคือ ไม่ว่า Data Model ภายใน Model Layer จะใช้เทคโนโลยีอะไร และมีการเปลี่ยนแปลงแหล่งข้อมูลอย่างไร ก็จะส่งผลกระทบต่อ Layer อื่นๆ น้อยมากหรืออาจไม่กระทบเลย

2.2.2 ชั้นแสดงผล (View Layer)

View Layer ควบคุมดูแลในส่วนของ User Interface ทำหน้าที่แสดงผล User Interface Component ผ่านทางหน้าจอ

2.2.3 ชั้นควบคุม (Controller Layer)

Controller Layer ควบคุมดูแลในส่วนของ Control Logic ทำหน้าที่จัดการกับเหตุการณ์ต่างๆ ที่ผู้ใช้งานกระทำกับ User Interface Component ผ่านทาง Interface, Control Page Flow และทำการติดต่อกับ Model Layer ตามเหตุการณ์ที่เกิดขึ้น

2.2.4 ข้อดีของ MVC Design Pattern

ความเป็นอิสระต่อกันของแต่ละ Layer ทำให้เมื่อทำการเปลี่ยนแปลงที่ Layer ใดๆ จะส่งผลกระทบต่อ Layer อื่นๆ น้อยมาก หรืออาจไม่เกิดผลกระทบใดๆเลย

1. ความยืดหยุ่นและความง่ายในการดูแลรักษา เนื่องจากการแบ่งชั้นที่เป็นอิสระต่อกัน ซึ่งต่างจากแนวทางในอดีตที่มีการฝังส่วนจัดการข้อมูลลงใน View Layer โดยตรง ทำให้ไม่สะดวกในการปรับปรุงแก้ไข และดูแล
2. เนื่องด้วย MVC เป็น Design Pattern หนึ่งที่ใช้สำหรับ Object-Oriented Programming ดังนั้น Model Layer ซึ่งทำหน้าที่ติดต่อกับข้อมูลจึงมีลักษณะของ Object ทุกประการ คือมีการ Encapsulation และ Inheritance ทำให้ผู้พัฒนาไม่จำเป็นต้องรู้การทำงานภายในก็สามารถพัฒนาได้ ทำให้การพัฒนาโปรแกรมแบบกลุ่มทำได้ง่าย และมีประสิทธิภาพมากยิ่งขึ้น
3. Controller Layer ใน MVC ทำหน้าที่แปลงเหตุการณ์ต่างๆ ที่เกิดขึ้นกับ User Interface Component ไปเป็น Method และ Data ซึ่งมีลักษณะคล้ายคลึงกับแนวทาง Event Driven Programming คือตัวโปรแกรมจะติดต่อสื่อสารกับผู้ใช้ผ่านทาง Event ที่ผู้ใช้กระทำกับ User Interface Component แต่ต่างกันตรงที่ใน Event Driven Programming จะใช้เวลาส่วนใหญ่ในการรอ Event และจะทำงานเมื่อเกิด Event เท่านั้น

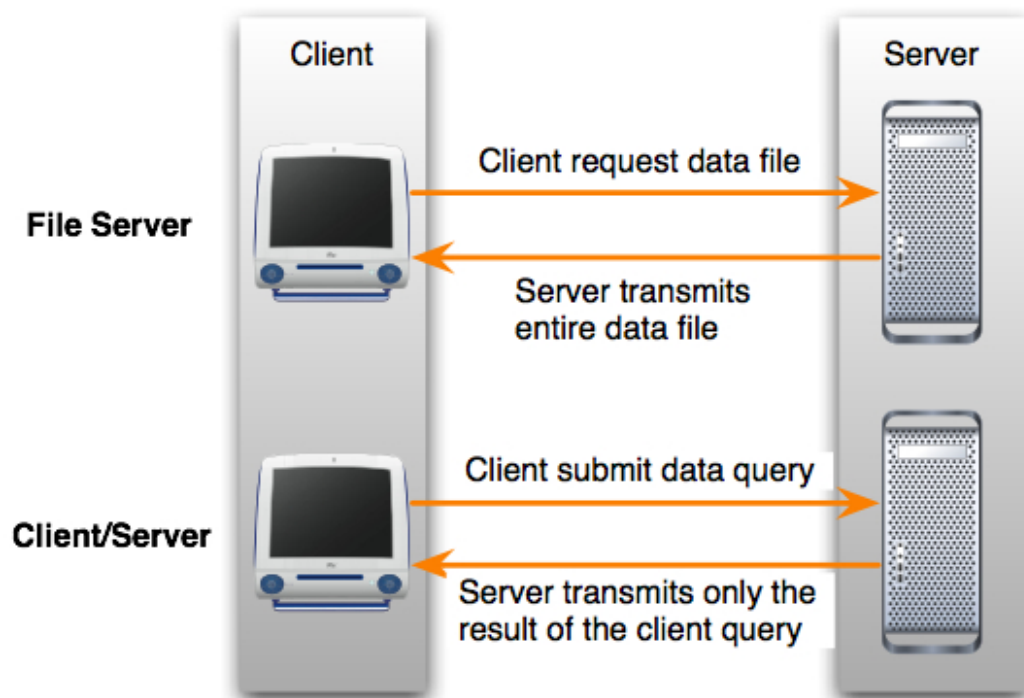
2.3 สถาปัตยกรรมแบบหลายชั้น (Multi-tier/n-tier Architecture)

สถาปัตยกรรมแบบหลายชั้น เป็นสถาปัตยกรรมแบบ Client-Server ซึ่งโปรแกรมถูกทำงานอยู่บน Software Agent มากกว่า 1 ตัว เช่นโปรแกรมที่ใช้ Middleware เพื่อให้บริการการร้องขอข้อมูลระหว่าง ผู้ใช้งาน และฐานข้อมูล ตัวอย่างของ Multi-tier Architecture ที่ถูกใช้กันอย่างแพร่หลายได้แก่ 3-tier Architecture

2.3.1 สถาปัตยกรรมแบบ 2 ชั้น (2-tier Architecture)

2-tier Architecture มักหมายถึงระบบที่แบ่งการประมวลผลออกเป็น Client (เครื่องลูกข่าย) และ Server (เครื่องแม่ข่าย) ซึ่งเชื่อมต่อกันโดยระบบเครือข่าย โดยทั่วไปแล้ว ในระบบแบบ 2-tier Architecture นั้น เครื่องลูกข่ายจะมีหน้าที่จัดการกับ User Interface ซึ่งประกอบไปด้วยข้อมูลที่ป้อนเข้าสู่ระบบ ข้อมูลที่เรียกดูจากระบบ และหน้าจอแสดงผล ในขณะที่เครื่องแม่ข่ายจะทำหน้าที่เก็บข้อมูล จัดเตรียมสภาวะแวดล้อมให้สามารถเข้าถึงข้อมูลได้ และการจัดการฐานข้อมูลต่างๆ

การติดต่อสื่อสารใน 2-tier Architecture นั้น เครื่องลูกข่ายจะส่งคำร้องขอไปยังเครื่องแม่ข่าย ซึ่งจะจัดหาข้อมูลแล้วส่งกลับมาให้เครื่องลูกข่าย โดย 2-tier Architecture สามารถแบ่งย่อยออกได้ตามลักษณะการส่งกลับข้อมูลของเครื่องแม่ข่ายดังนี้

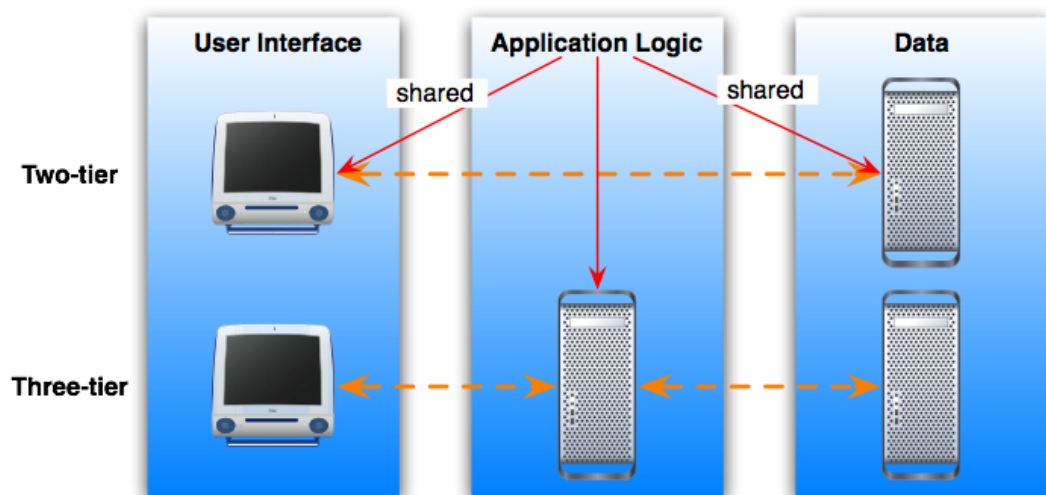


รูปที่ 2.3 สถาปัตยกรรมแบบ 2 ชั้น

1. File Server Design การออกแบบระบบแบบนี้ เครื่องแม่ข่ายจะส่งกลับข้อมูลทั้งหมดไปให้กับเครื่องลูกข่าย
2. Client-Server Design การออกแบบระบบแบบนี้ เครื่องแม่ข่ายจะส่งกลับข้อมูลเฉพาะส่วนที่ถูกร้องขอไปให้กับเครื่องลูกข่าย

2.3.2 สถาปัตยกรรมแบบ 3 ชั้น (3-tier Architecture)

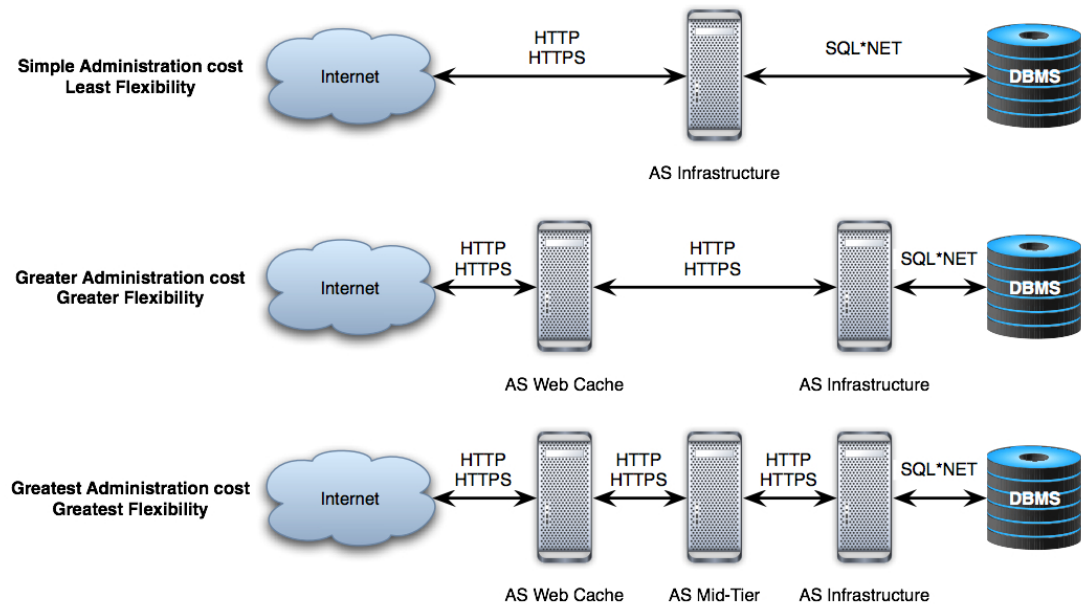
3-tier Architecture คือระบบที่มีการแบ่งการประมวลผลออกเป็น 3 ระดับได้แก่ Client (เครื่องลูกข่าย) Application Server (เครื่องแม่ข่ายโปรแกรมประยุกต์) Data Server (เครื่องแม่ข่ายข้อมูล) ซึ่งในระบบแบบ 3-tier Architecture นี้ User Interface จะแสดงที่เครื่องลูกข่าย และข้อมูลถูกจัดเก็บไว้ที่เครื่องแม่ข่ายข้อมูลคล้ายคลึงกับใน 2-tier Architecture แต่จะใน 3-tier Architecture จะมีการเพิ่มเติมในส่วน of เครื่องแม่ข่ายโปรแกรมประยุกต์ขึ้นมา เพื่อดูแลจัดการการร้องขอของเครื่องลูกข่าย และแปลคำร้องขอเหล่านั้นให้อยู่ในรูปของคำสั่งการเข้าถึงข้อมูลส่งไปยังเครื่องแม่ข่ายข้อมูลดังรูป



รูปที่ 2.4 สถาปัตยกรรมแบบ 3 ชั้น

2.3.3 สถาปัตยกรรมแบบหลายชั้น (Multi/n-tier Architecture)

Multi-tier Architecture คือระบบที่พัฒนาต่อมาจาก 3-tier Architecture โดยแบ่งแยก Application Server ออกมาเป็นชั้นย่อยๆ ตามความต้องการของระบบ ดังตัวอย่างในรูป



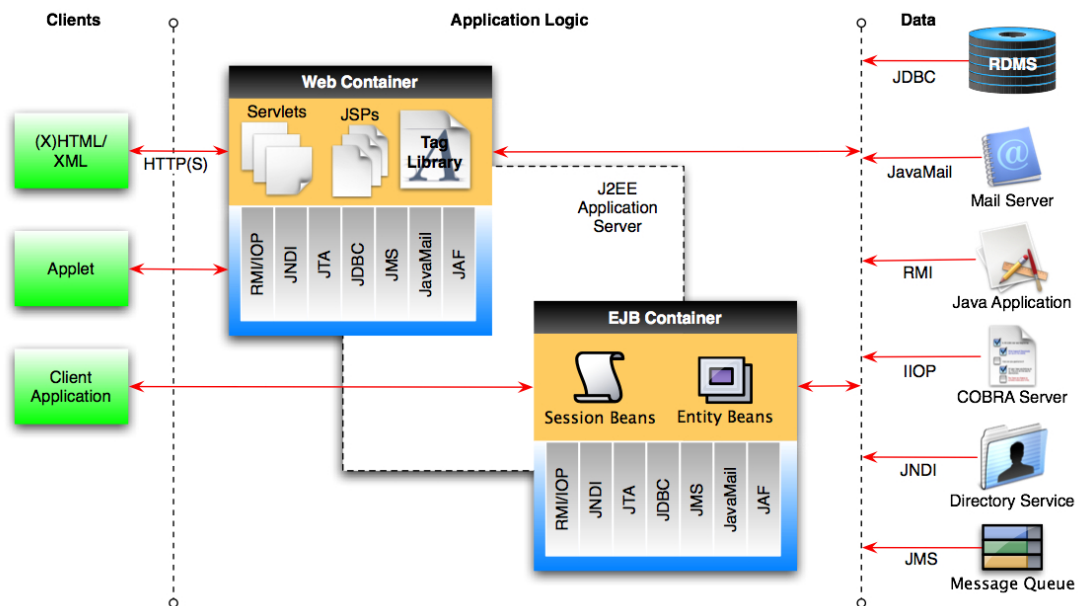
รูปที่ 2.5 สถาปัตยกรรมแบบหลายชั้น

2.3.4 ข้อดีสถาปัตยกรรมแบบหลายชั้น

1. มีความปลอดภัยสูง เนื่องจากผู้ใช้ไม่สามารถเข้าถึงข้อมูลภายในฐานข้อมูลได้โดยตรง
2. มีความอิสระต่อกันสูง เนื่องจากมีการแยกออกเป็นชั้นต่างๆ ที่ทำงานแยกจากกัน ดังนั้นหากมีความผิดพลาดในชั้นใดก็จะไม่ส่งผลกระทบต่อข้อมูลในชั้นอื่นๆ หรืออาจเกิดน้อยมาก
3. รองรับการขยายตัวได้เป็นอย่างดี เนื่องจากมีการแบ่งงานที่ต้องทำออกเป็นชั้นต่างๆ ทำให้ระบบสามารถรองรับการใช้งานได้เพิ่มมากขึ้นและสนับสนุนเทคโนโลยีที่หลากหลายมากขึ้นด้วยเช่นกัน
4. มีประสิทธิภาพสูง เนื่องจากในแต่ละชั้นจะดูแลจัดการเฉพาะงานที่ตนเองรับผิดชอบ ไม่ต้องไปยุ่งกับชั้นอื่นๆ

2.4 Java 2 Enterprise Edition (J2EE) Architecture

Java 2 Platform, Enterprise Edition (J2EE) เป็น Platform ที่เป็น Superset ของ Java 2 Standard Edition (J2SE) ซึ่งมีการจัดเตรียมเครื่องมือและเทคโนโลยีให้เหมาะสมกับการพัฒนาและนำเอา Enterprise Application มาใช้งานแบบ Multi-tier Architecture



รูปที่ 2.6 สถาปัตยกรรมภายในของ Java 2 Enterprise Edition (J2EE)

เทคโนโลยีและข้อกำหนดที่น่าสนใจใน J2EE ได้แก่

2.4.1 Servlet

Servlet คือ Java Class ที่ถูกเก็บและทำงานบน Application Server และเรียกใช้ความสามารถของเครื่อง Server ผ่านทางบริการพิเศษของ Application Server

HTTP Servlet คือ Servlet ประเภทหนึ่งที่มีการร้องขอจาก Client Web Browser ผ่านทาง HTTP Data Stream และตอบกลับการร้องขอนั้นไปยัง Web Browser นั้น

2.4.2 JavaServer Page (JSP)

JavaServer Page (JSP) เป็นเทคโนโลยีที่แตกย่อยออกมาจาก Servlet เนื่องจากในการใช้งาน Servlet นั้นค่อนข้างยุ่งยาก ทั้งในขั้นตอนการเขียนโปรแกรม และการออกแบบ User Interface ดังนั้น JSP จึงเข้ามาตอบโจทย์ตรงส่วนนี้ได้เป็นอย่างดี โดยลักษณะของ JSP จะคล้ายคลึงกับโครงสร้างของ Hyper Text Markup Language (HTML) ดังนั้นเวลาใช้งานส่วนมากแล้วจะนำเอา JSP ฝังเข้าไปในส่วนโค้ด HTML และเนื่องจาก JSP แตกย่อยออกมาจาก Servlet ดังนั้นเมื่อมีการร้องขอแสดงผล โค้ด JSP ในหน้านั้นจะถูกแปลงให้อยู่ในรูปของ Servlet แล้วส่งไปทำงานบน Application Server อีกที

2.4.3 JavaBeans

JavaBeans คือ Java Component ซึ่งเขียนขึ้นมาโดยใช้ภาษา Java ทั้งหมด มีการทำงานที่ฝั่ง Server Side เหมือนกับ JSP มีการทำงานบน JVM (Java Virtual Machine)

2.4.4 ข้อดีของ JSP

1. สามารถทำงานได้บนทุกๆ ระบบปฏิบัติการ โดยที่ไม่ต้องทำการ Compile ใหม่ทุกครั้ง
2. ผลลัพธ์ของ JSP นั้นเป็น HTML มาตรฐาน ซึ่งสามารถแสดงผลได้บน Web Browser ทั่วไป
3. ไม่จำเป็นต้องมี Java Virtual Machine ที่ฝั่ง Client
4. ง่ายและสะดวกในการออกแบบ User Interface ให้มีความสวยงามและหลากหลาย

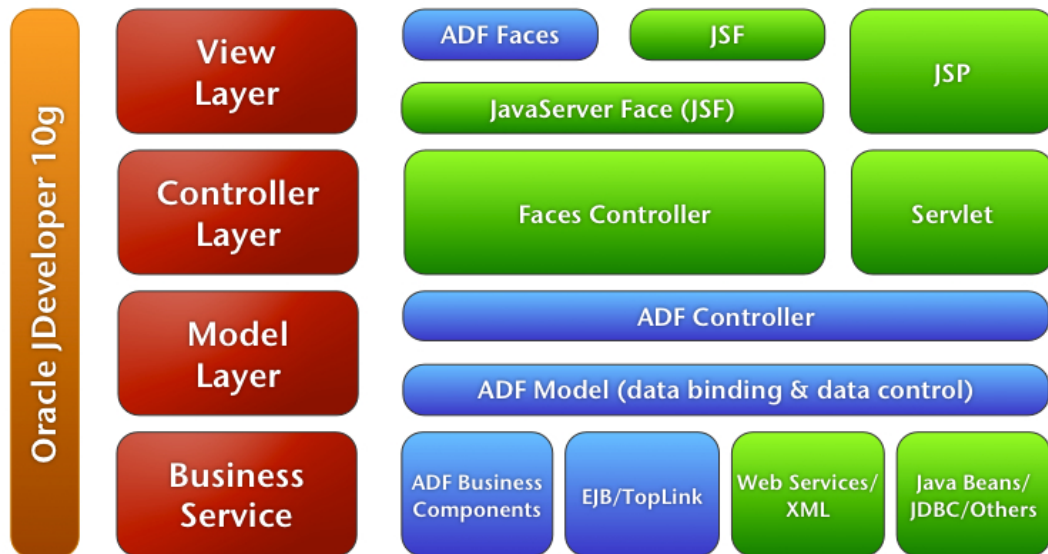
2.4.5 ข้อดีของ JavaBeans

1. สามารถทำงานได้บนทุกๆ ระบบปฏิบัติการ โดยที่ไม่ต้องทำการ Compile ใหม่ทุกครั้ง
2. มีการ load ลงหน่วยความจำเพียงครั้งเดียวจนกว่าจะมีการเลิกใช้งานโปรแกรม
3. สามารถนำ Component ที่เขียนมาแล้ว กลับมาใช้ใหม่ได้อีก
4. สามารถนำความรู้ที่ใช้ในภาษา Java มาใช้ในการเขียนโปรแกรมร่วมด้วยได้

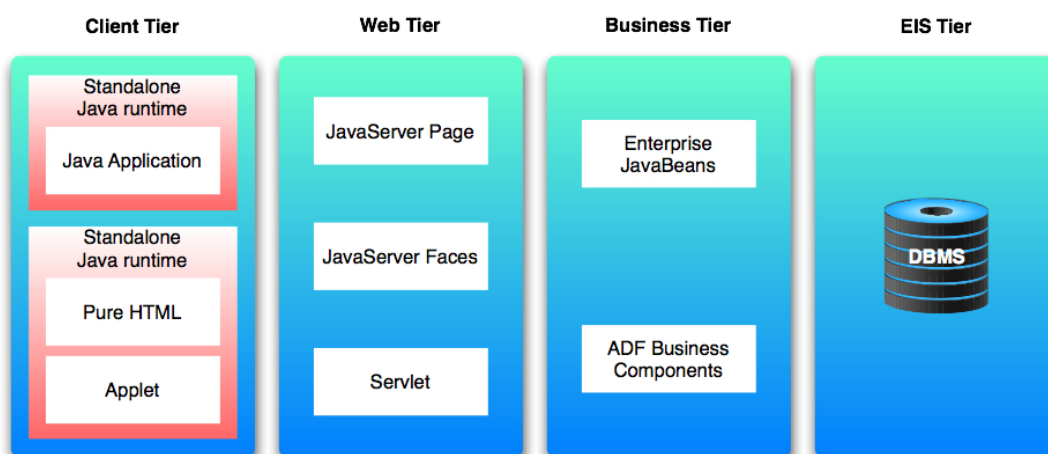
2.5 Oracle Application Development Framework (OADF) Architecture

Oracle Application Development Framework เป็นกรอบการทำงานตาม Model-View-Controller Design Pattern ที่มีเครื่องมือสำหรับผู้พัฒนา Application ด้วย Java 2 Enterprise Edition (J2EE) ครอบคลุมทุกความสามารถตามมาตรฐานของ Sun Microsystems โดย Oracle Application Development Framework จะช่วยลดขั้นตอนในการออกแบบ พัฒนาและนำ Application ไปใช้งาน

Oracle ADF ประกอบไปด้วยชั้นย่อยๆ ตาม MVC Design Pattern อันได้แก่ ชั้น Model, ชั้น View, ชั้น Controller และชั้น Business Logic



รูปที่ 2.7 สถาปัตยกรรมของ Oracle Application Development Framework



รูปที่ 2.8 สถาปัตยกรรมของ Oracle Application Development Framework กับ J2EE

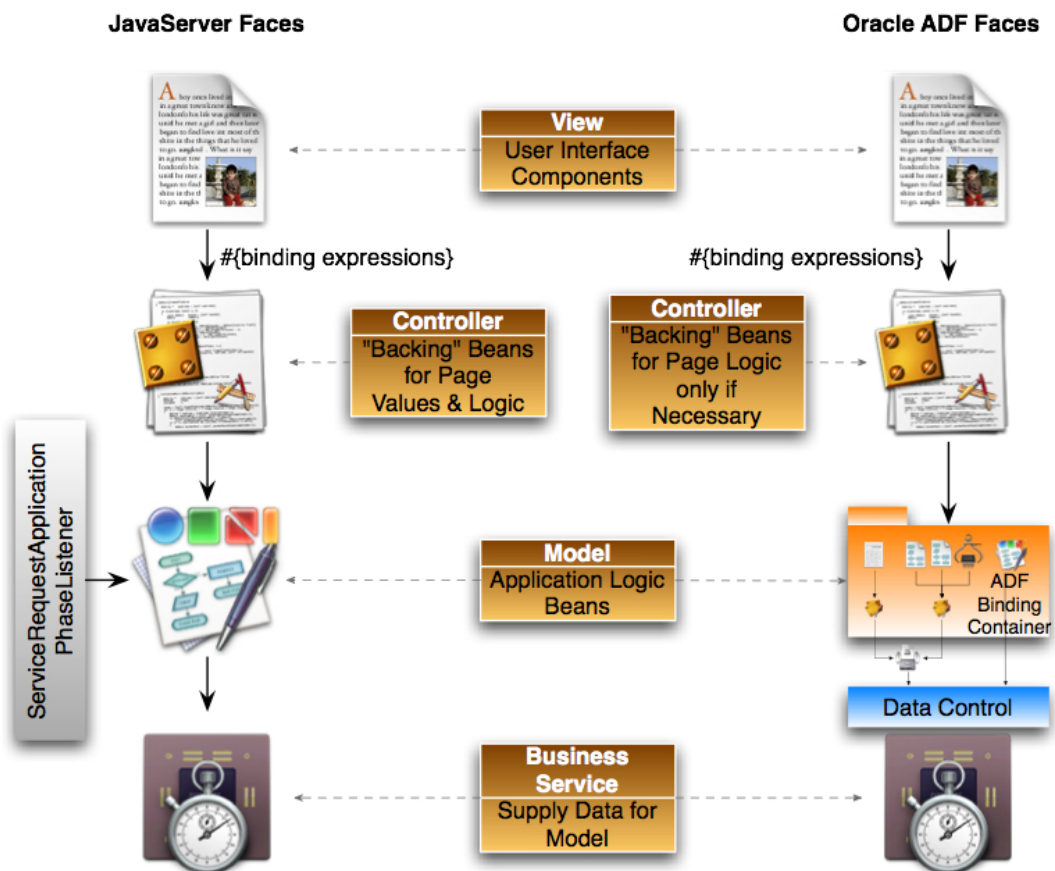
2.5.1 Model Layer

Model Layer คือเลเยอร์ที่รับผิดชอบในการเชื่อมต่อการทำงานระหว่าง View, Controller Layer เข้ากับ Business Service Layer เพื่อให้สามารถทำงานร่วมกันกับเทคโนโลยีที่หลากหลายใน Business Service Layer ได้ โดยใน Model Layer ของ Oracle ADF นี้จะเป็นไปตามมาตรฐาน JSR 227 (มาตรฐานการเชื่อมต่อและเข้าถึงข้อมูลสำหรับ J2EE)

2.5.2 View Layer

View Layer ดูแลรับผิดชอบเกี่ยวกับ User Interface และการแสดงผลข้อมูล โดยใน View Layer ของ Oracle ADF สนับสนุนการทำงานร่วมกันกับเทคโนโลยีต่างๆ ดังต่อไปนี้

1. เทคโนโลยีสำหรับ Rich Client Application เป็น Application ที่ติดตั้งในเครื่องคอมพิวเตอร์ของผู้ใช้งาน
 - a) Swing/JClient - Swing เป็นเทคโนโลยีในการแสดงผลสำหรับ Application ทั่วไป ที่เรียกใช้ Swing Library ของ Java
2. เทคโนโลยีสำหรับ Web Application



รูปที่ 2.9 การทำงานของ JavaServer Faces เทียบกับ Oracle ADF Faces

- a) JavaServer Page (JSP) เป็นภาษาสคริปต์ที่ใช้ในการแสดงผล Dynamic Web Page โดยมีพื้นฐานมาจาก Servlet
- b) Oracle UIX (User Interface XML) เป็นเทคโนโลยีของ Oracle ในการแสดงผลด้วยการนำเอา XML มาประยุกต์เพื่อใช้งานในการออกแบบ User Interface เพื่อให้ทำงานได้อย่างมีประสิทธิภาพมากยิ่งขึ้น
- c) JavaServer Faces (JSF) เป็นมาตรฐานใหม่ในการพัฒนา Web Application ด้วยเทคโนโลยี Java ซึ่ง JSF ทำให้การพัฒนา User

Interface ของ Java Web Application เป็นเรื่องง่ายขึ้น ด้วยวิธีการแบบ Component-centric นอกจากนี้ JSF สร้างขึ้นบนพื้นฐานของ MVC Design Pattern ทำให้การดูแล ปรับปรุงแก้ไขทำได้ง่ายอีกด้วย

- d) Oracle ADF Faces Oracle ADF Faces เป็นหนึ่งในเทคโนโลยีทางเลือกสำหรับ View Layer ใน Oracle ADF ซึ่งสร้างมาจากพื้นฐานของ JavaServer Faces (JSF) โดยทาง Oracle ได้รวมเอาข้อดีของ JSF เข้ากับ Oracle UIX เข้าไว้ในตัวของ Oracle ADF Faces นี้เอง

2.5.3 Controller Layer

Controller Layer ดูแลรับผิดชอบในส่วนของการควบคุมและวิธีการทำงานของส่วนต่างๆ ใน Application บนเว็บ โดยใน Controller Layer ของ Oracle Application Development Frame-work สนับสนุนการทำงานร่วมกันกับเทคโนโลยีต่างๆ ดังต่อไปนี้

1. Struts เป็นมาตรฐานแบบเปิดสำหรับการพัฒนา Application บนเว็บด้วย J2EE โดย Struts นั้นเป็นส่วนเสริมของ Java Servlet ที่ช่วยให้การพัฒนาโปรแกรมตาม MVC Design Pattern เป็นไปได้อย่างสะดวกสบายมากยิ่งขึ้น โดยส่วนที่เป็น Controller ของ Struts จะจัดการโดย HTTP Servlet ซึ่งทำงานตาม XML configuration file
2. JavaServer Faces (JSF) ใน Oracle Application Development Framework นั้น JSF ครอบคลุมการทำงานลงมาถึงในชั้น Controller โดยบทบาทของ JSF ในชั้น Controller นั้นมีความคล้ายคลึงกับ Struts เป็นอย่างมาก เพียงแต่ JSF นั้นจะได้เปรียบในส่วนของการจัดการกับ User Interface ได้ดีกว่ามาก เนื่องจาก JSF นั้นอ้างอิงการกำหนดบทบาทของส่วนประกอบบนหน้าเว็บคล้ายกับที่ใช้ใน Application แบบที่เรียกใช้ Swing Library

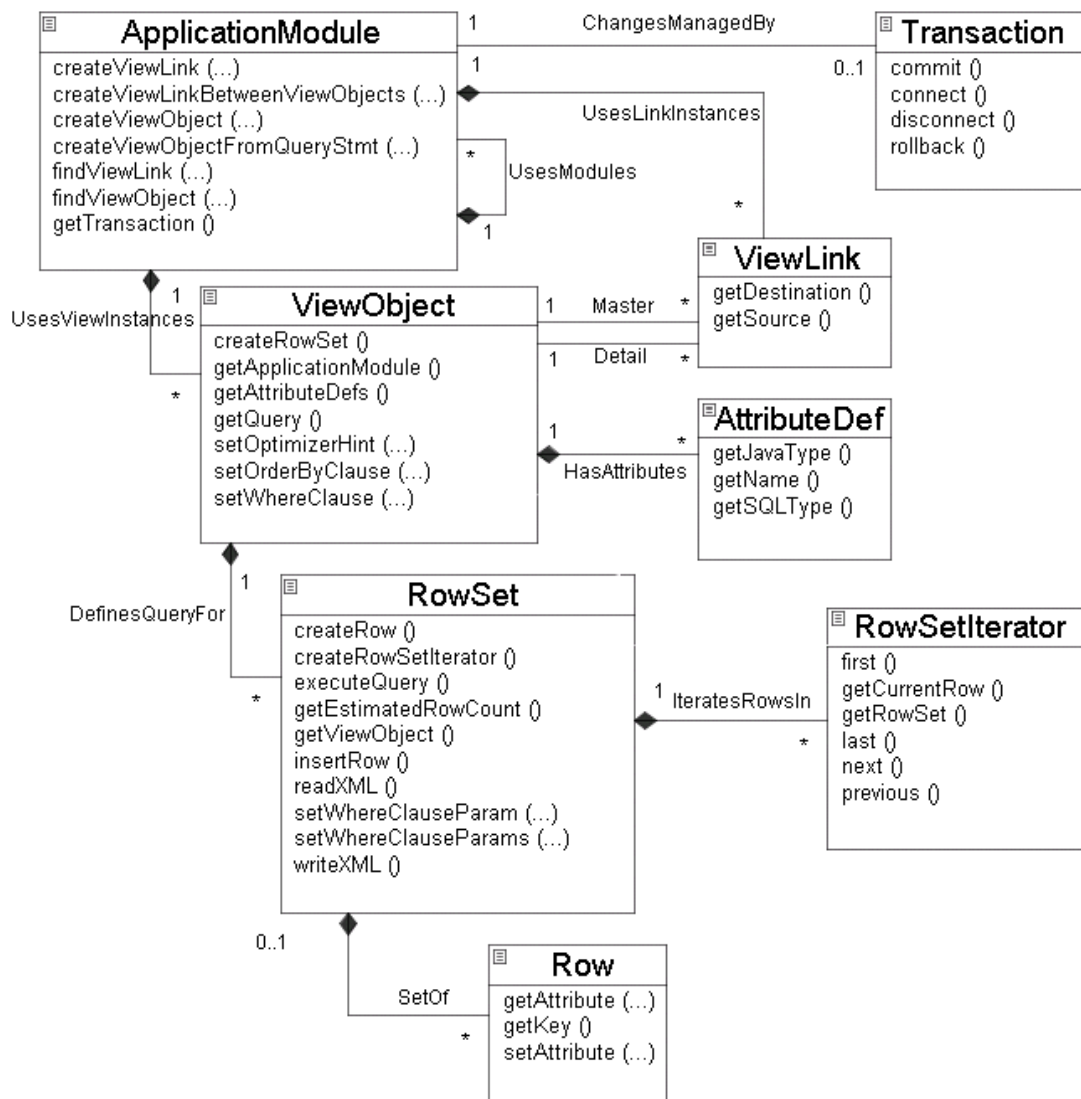
2.5.4 Business Service Layer

Business Service Layer ดูแลจัดการการเข้าถึงข้อมูลจากหลายๆ แหล่งและดูแลจัดการกับ Business logic โดยใน Business Service Layer ของ Oracle ADF นี้สนับสนุนเทคโนโลยีที่หลากหลายได้แก่

1. Web Service เป็นรูปแบบหนึ่งของการสร้าง Reuse Component ซึ่งถูกเรียกใช้งานจากภายในหรือจากโปรแกรมภายนอกที่ได้รับอนุญาต
2. Java Beans เป็น Library ของ Class ที่ถูกสร้างขึ้นเพื่อนำไปใช้งานต่างๆ และสามารถนำมาใช้ในการ reuse component เพื่อช่วยลดระยะเวลาในการออกแบบและพัฒนาโปรแกรมได้ โดยผู้พัฒนาโปรแกรมสามารถปรับเปลี่ยนคุณสมบัติ

บางอย่างของ Java Beans นั้นๆ เพื่อให้เหมาะสมกับโปรแกรมที่ต้องการนำไปใช้ได้

3. EJB Session Beans (Enterprise Java Beans Session Beans) คล้ายกับ Java Beans แตกต่างกันในข้อกำหนดบางประการเช่น Java Beans นั้นทำงานบน JVM (Java Virtual Machine) ในขณะที่ EJB นั้นทำงานบน EJB Container ของ J2EE Server เท่านั้นเป็นต้น
4. Oracle ADF Business Component เป็นกรอบการทำงานที่สร้างด้วย XML บน Java Platform ซึ่งช่วยเพิ่มประสิทธิภาพ และความยืดหยุ่นในการพัฒนาและติดตั้ง Multi-tier Application ที่มีการใช้งาน Database ค่อนข้างมาก โดยการสร้าง Object ที่สามารถใช้งานซ้ำได้สำหรับกิจกรรมต่างๆ ที่กระทำกับ Database



รูปที่ 2.10 การทำงานของ Oracle ADF Business Component

Oracle ADF Business Component ประกอบไปด้วยส่วนย่อยๆ มากมายเช่น การแปลง Object-relational, การจัดการ Transaction, การทำ Locking และการประกาศ Validation Rule เป็นต้น โดย Oracle ADF Business Component จะประกอบไปด้วยส่วนย่อยๆ ดังต่อไปนี้

- a) Entity Object เทียบได้กับ Entity Type ในการออกแบบ Database เป็นโครงสร้างพื้นฐานที่สุดของ Oracle ADF Business Components ซึ่งทำหน้าที่จัดการการ Encapsulate Business Logic และติดต่อกับ Database สำหรับแต่ละ Entity ใน Business Model นอกจากนี้เรายังสามารถกำหนด Validation Logic ลงใน Java Implementation Class ของ Entity Object เพื่อกำหนดให้ตัว ADF จัดการเกี่ยวกับการตรวจสอบความถูกต้องของข้อมูลและ Logic ต่างๆตามที่ได้กำหนดไว้โดยอัตโนมัติ
- b) Association เทียบได้กับ Foreign Key ในการออกแบบ Database เป็นการเชื่อมโยง Entity Object 2 Entity เข้าด้วยกัน เพื่อเชื่อมโยง และแสดงความสัมพันธ์ของข้อมูลระหว่าง Entity
- c) Validation Rule เป็น JavaBeans ซึ่งถูกสร้างขึ้นเพื่อใช้สำหรับตรวจสอบความถูกต้องของข้อมูล และ Logic บน Entity Object โดยเราจะสร้าง Java Implementation Class ของ Entity Object เพื่อกำหนดว่าต้องการตรวจสอบอะไร แล้วใช้ Component Definition ซึ่งเป็น XML เพื่อนำเอา Validation Rule ที่ได้ไปใช้กับ Entity Object ที่ต้องการ
- d) Domain เทียบได้กับ Data Type ของ Attribute ในการออกแบบ Database โดย Domain จะมี Validation Constructor สำหรับตรวจสอบความถูกต้องของข้อมูลที่ป้อนเข้าไป
- e) View Object ภายใน Oracle ADF Business Components มีความสามารถดังต่อไปนี้
 - I. สร้าง View สำหรับข้อมูลทางธุรกิจได้หลากหลายจาก Entity Object เพื่อใช้สำหรับการสร้าง Business Logic โดยไม่ต้องไปยุ่งกับตัว Entity Object โดยตรง
 - II. สามารถใช้งานคำสั่ง SQL ตามมาตรฐาน SQL92 เพื่อควบคุมการสร้าง View ได้ตามต้องการ
 - III. ให้ Database จัดการ Join ตารางที่เกี่ยวข้องแทนการเขียน Code ฝังลงใน Application ซึ่งจะช่วยให้ข้อมูลจากหลายๆ Entity สามารถถูกเรียกใช้ได้ในครั้งเดียว

- f) View Link เป็นวิธีการสร้างความสัมพันธ์ระหว่าง View แบบ Master-Detail โดย ADF จะทำหน้าที่จัดการข้อมูลภายในของ Detail View ให้ตรงกับ Master-View ให้อัตโนมัติ
- g) ApplicationModule เป็นการสร้าง Data Model ขึ้นมาสำหรับนำไปใช้งานใน Application โดยภายใน ApplicationModule จะประกอบไปด้วยข้อมูลต่างๆ และ Method ที่จำเป็นในการติดต่อและจัดการกับ Database